



先進工学研究科 電子システム工学専攻

音響工学特論

第5回

植木 祥高

東京理科大学 先進工学部
電子システム工学科

授業計画

- #01 イン트로ダクション
- #02 機械学習の歴史と動向
- #03 機械学習分野の問題・具体的手法
- #04 Pythonのプログラミング基礎
- #05 音響収録の基礎**
- #06 ニューラルネットワークの基礎 ①
- #07 ニューラルネットワークの基礎 ②
- #08 ニューラルネットワークの実装 ①
- #09 ニューラルネットワークの実装 ②
- #10 ニューラルネットワークの実装 ③
- #11 ディープラーニングの概要と動向
- #12 音響データを用いた機械学習 ①
- #13 音響データを用いた機械学習 ②
- #14 プレゼンテーション
- #15 到達度評価
(機械学習・音響工学の知識と習得度を評価)

※ 評価：60%

※ 評価：40%

音響収録の基礎

- 音響データの取得
- Pythonを使ったデータ解析
- 周波数解析

音響データの取得

- スマートフォンのマイクを活用
(iPhone, Androidの利用を推奨)

App Storeレビュー

アプリを購入またはダウンロードするにはMac App Storeを開いてください。



PCM録音 (4+)
高音質ボイスレコーダー
Kohei YASUI
iPhone対応
「ビジネス」内47位
★★★★★ 4.5 + 6,276件の評価
無料

iPhoneスクリーンショット



Google Play

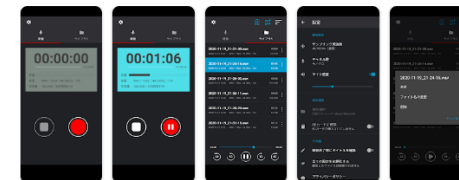
PCM録音

Kohei YASUI
広告が表示されます

4.1★ | 100万+ | 1.17万 件のレビュー | ダウンロード | 全ユーザー対象 ①

インストール

このアプリは、お使いのデバイスで利用できません



このアプリについて →

最高音質・長時間録音可能なボイスレコーダーです。
録音は、ロスレス圧縮のリニアPCM(WAV)形式と非可逆圧縮のAAC形式を選択可能です。

バックグラウンドでの長時間録音にも対応。
サンプリングレートは目的に合わせ8k, 16k, 44.1k, 48kHzに変更可能(AAC形式で...

アプリのサポート

類似のアプリ

- 音声ボイスレコーダー
Digipom
4.7★
- Riverside: Record podcasts
Riverside.fm
4.6★
- サウンド&ボイスレコーダー - AS
NLL APPS
4.4★
- 通話レコーダー - Talker ACR
Joicoo Limited
3.8★

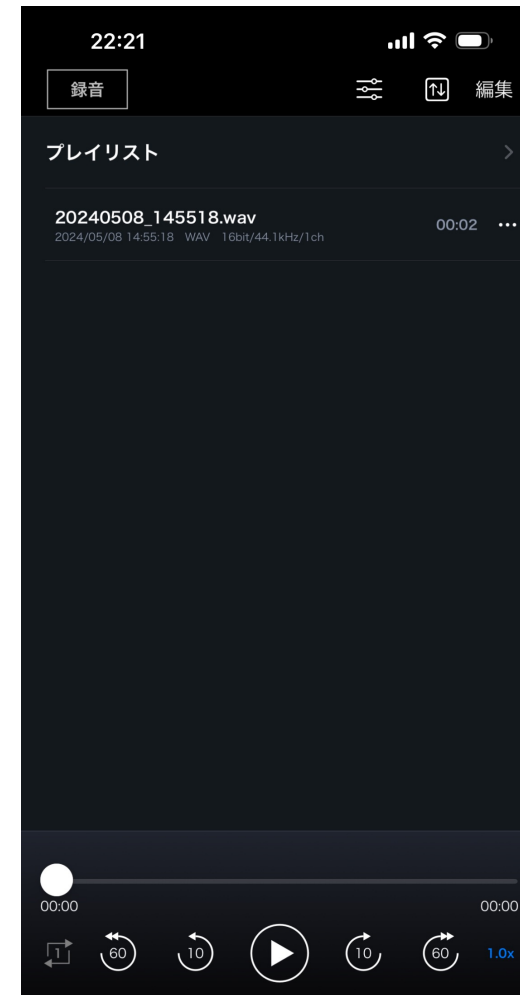
PCM録音

- 録音設定



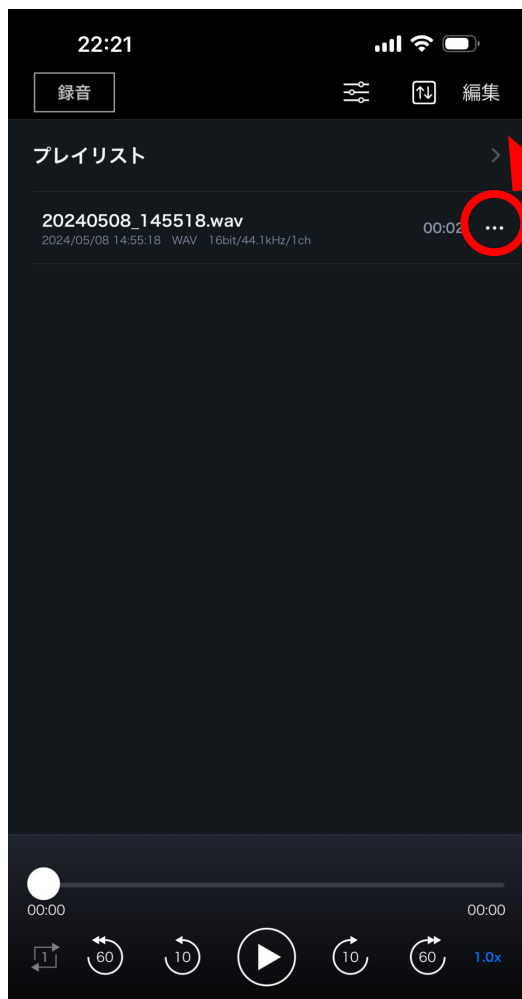
PCM録音

- 録音



PCM録音

- 録音データの確認 & 転送



※ OneDriveに転送

音響データの読み込み

- Pythonライブラリ
Pydub, librosa, soundfile

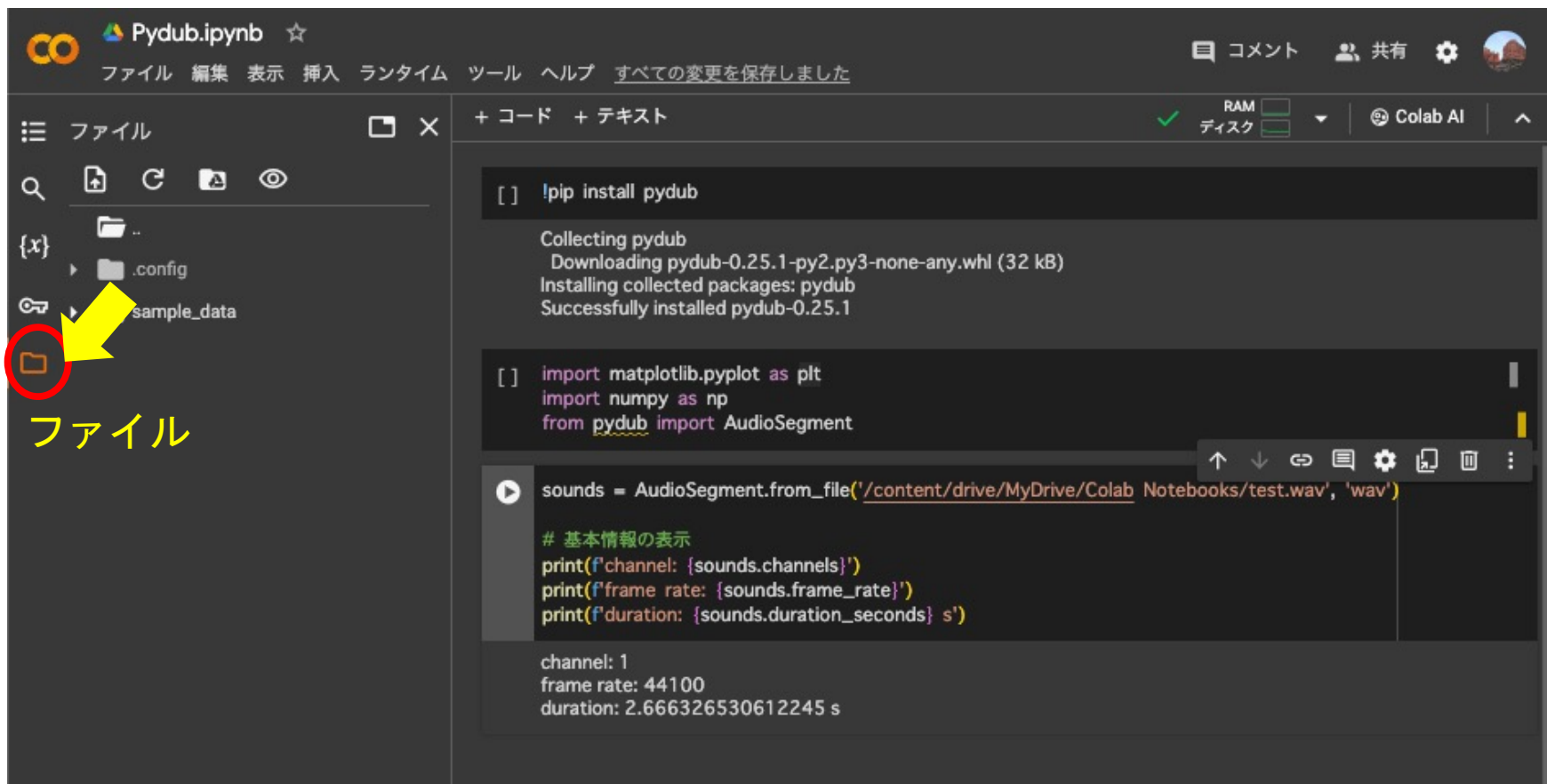
※ 今回はPydubを取り上げる

- Pydub
Google Colabにデフォルトでインストールされていない。
コードセル：!pip install pydub

```
[ ] !pip install pydub  
  
Collecting pydub  
  Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)  
Installing collected packages: pydub  
Successfully installed pydub-0.25.1
```


Google Driveに転送したデータをPydubで読み込んでみよう

Google ColabにGoogle Driveをマウントする



Pydub.ipynb ☆

ファイル 編集 表示 挿入 ランタイム ツール ヘルプ すべての変更を保存しました

RAM ディスク Colab AI

ファイル

sample_data

ファイル

```
[ ] !pip install pydub
```

Collecting pydub
Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Installing collected packages: pydub
Successfully installed pydub-0.25.1

```
[ ] import matplotlib.pyplot as plt
import numpy as np
from pydub import AudioSegment
```

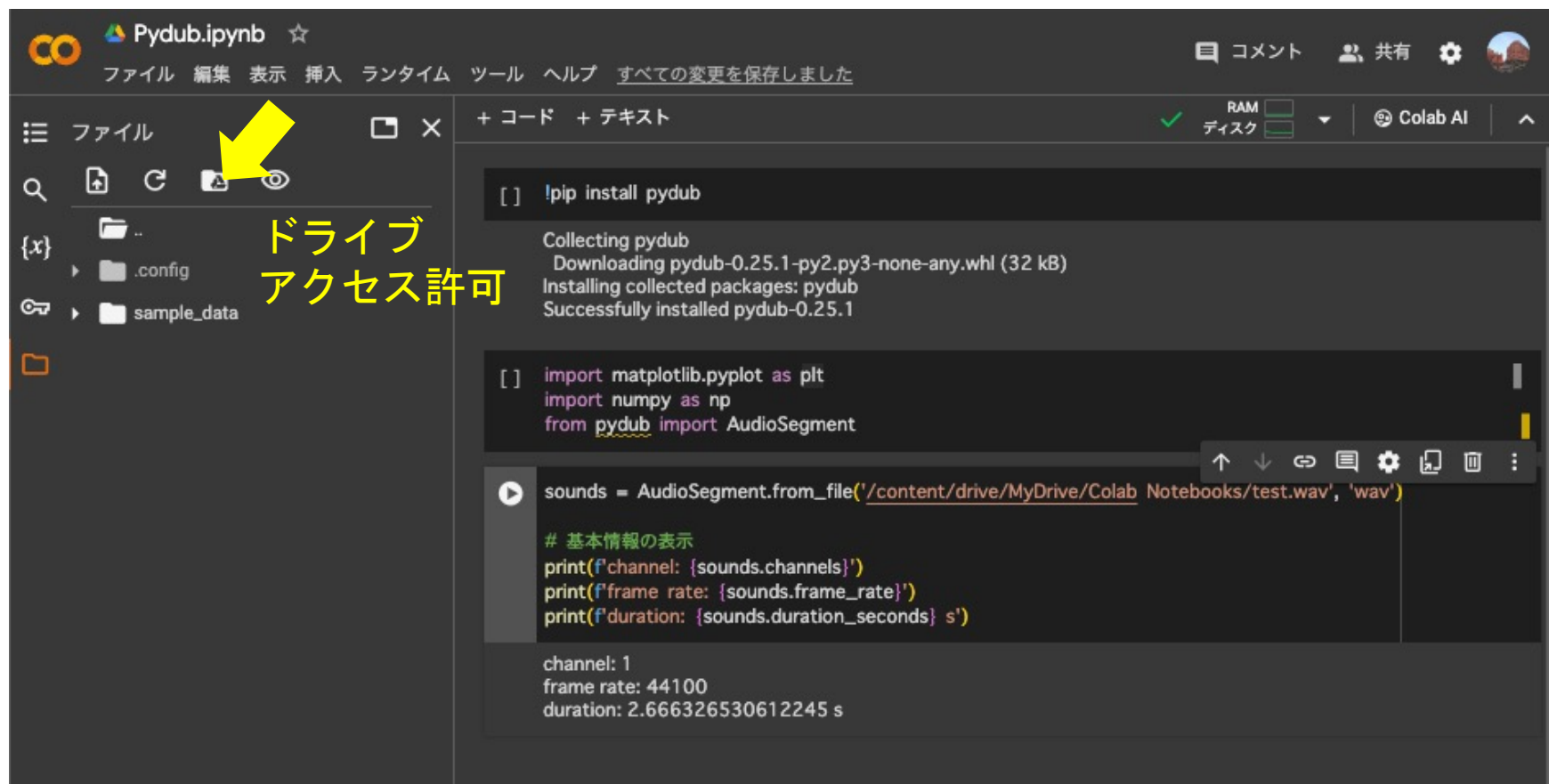
```
sounds = AudioSegment.from_file('/content/drive/MyDrive/Colab Notebooks/test.wav', 'wav')

# 基本情報の表示
print(f'channel: {sounds.channels}')
print(f'frame rate: {sounds.frame_rate}')
print(f'duration: {sounds.duration_seconds} s')
```

channel: 1
frame rate: 44100
duration: 2.666326530612245 s

Google Driveに転送したデータをPydubで読み込んでみよう

Google ColabにGoogle Driveをマウントする



Pydub.ipynb ☆

ファイル 編集 表示 挿入 ランタイム ツール ヘルプ すべての変更を保存しました

RAM ディスク Colab AI

ドライブ アクセス許可

```
[ ] !pip install pydub

Collecting pydub
  Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Installing collected packages: pydub
Successfully installed pydub-0.25.1
```

```
[ ] import matplotlib.pyplot as plt
import numpy as np
from pydub import AudioSegment

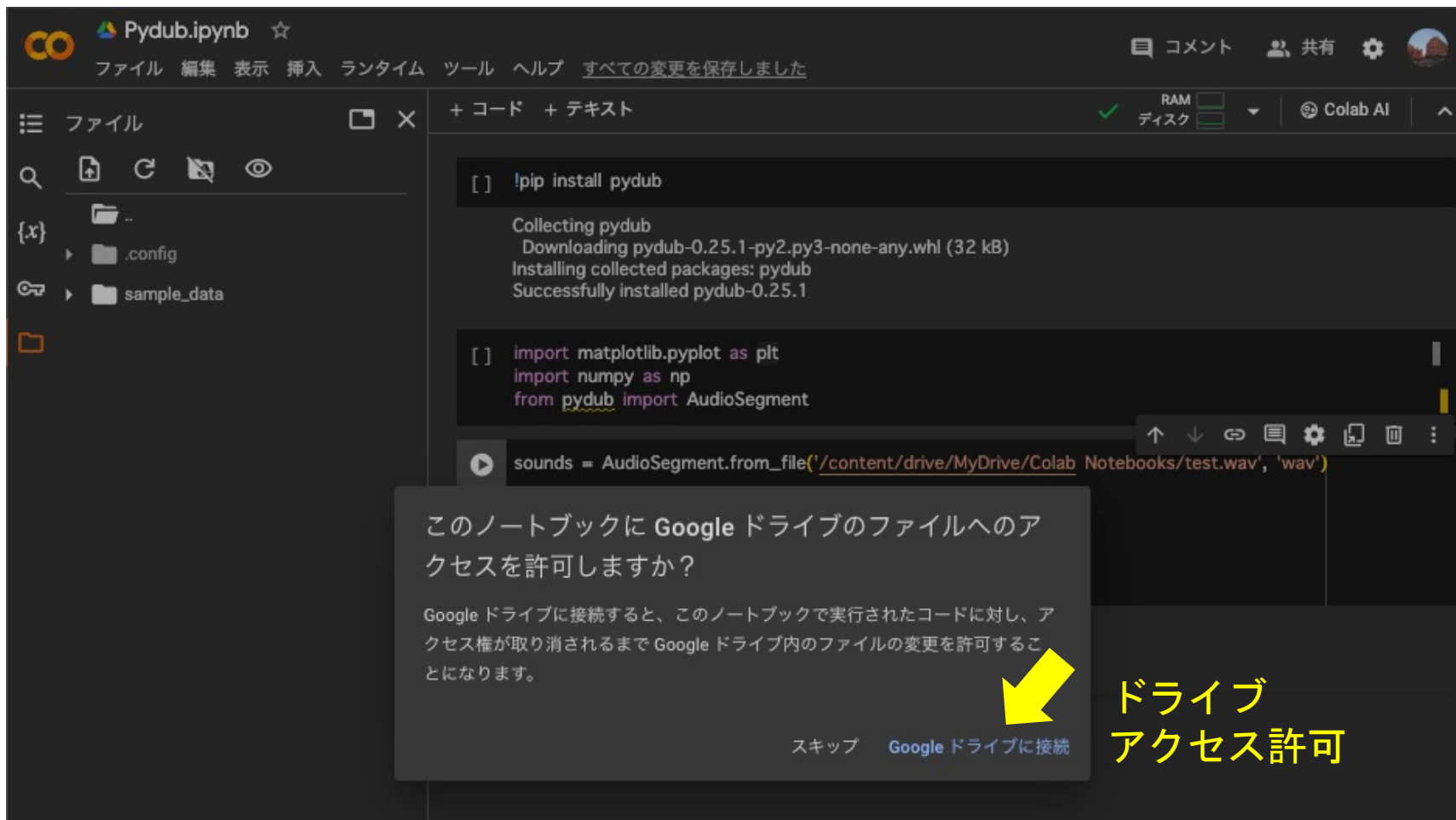
sounds = AudioSegment.from_file('/content/drive/MyDrive/Colab Notebooks/test.wav', 'wav')

# 基本情報の表示
print(f'channel: {sounds.channels}')
print(f'frame rate: {sounds.frame_rate}')
print(f'duration: {sounds.duration_seconds} s')
```

channel: 1
frame rate: 44100
duration: 2.666326530612245 s

Google Driveに転送したデータをPydubで読み込んでみよう

Google ColabにGoogle Driveをマウントする



Pydub.ipynb ☆

ファイル 編集 表示 挿入 ランタイム ツール ヘルプ すべての変更を保存しました

RAM ディスク Colab AI

+ コード + テキスト

```
[ ] !pip install pydub
```

Collecting pydub
Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Installing collected packages: pydub
Successfully installed pydub-0.25.1

```
[ ] import matplotlib.pyplot as plt
import numpy as np
from pydub import AudioSegment
```

```
sounds = AudioSegment.from_file('/content/drive/MyDrive/Colab Notebooks/test.wav', 'wav')
```

このノートブックに Google ドライブのファイルへのアクセスを許可しますか？

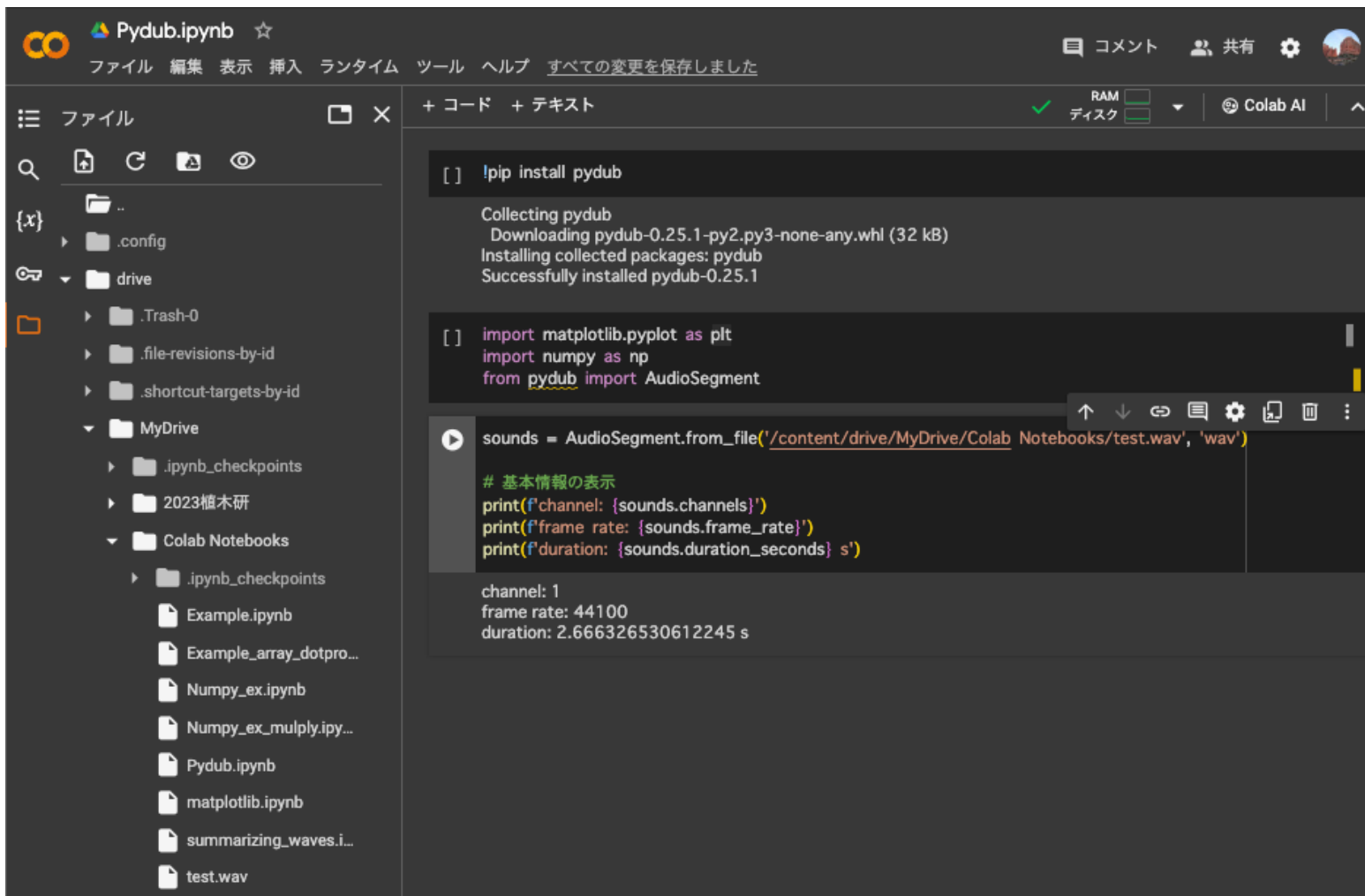
Google ドライブに接続すると、このノートブックで実行されたコードに対し、アクセス権が取り消されるまで Google ドライブ内のファイルの変更を許可することになります。

スキップ Google ドライブに接続

ドライブ
アクセス許可

Google Driveに転送したデータをPydubで読み込んでみよう

Google ColabにGoogle Driveマウント完了後



The screenshot shows the Google Colab interface with a notebook titled "Pydub.ipynb". The left sidebar displays the file explorer with a "drive" folder mounted. The main code area contains the following code:

```
[ ] !pip install pydub

Collecting pydub
  Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Installing collected packages: pydub
Successfully installed pydub-0.25.1

[ ] import matplotlib.pyplot as plt
import numpy as np
from pydub import AudioSegment

sounds = AudioSegment.from_file('/content/drive/MyDrive/Colab Notebooks/test.wav', 'wav')

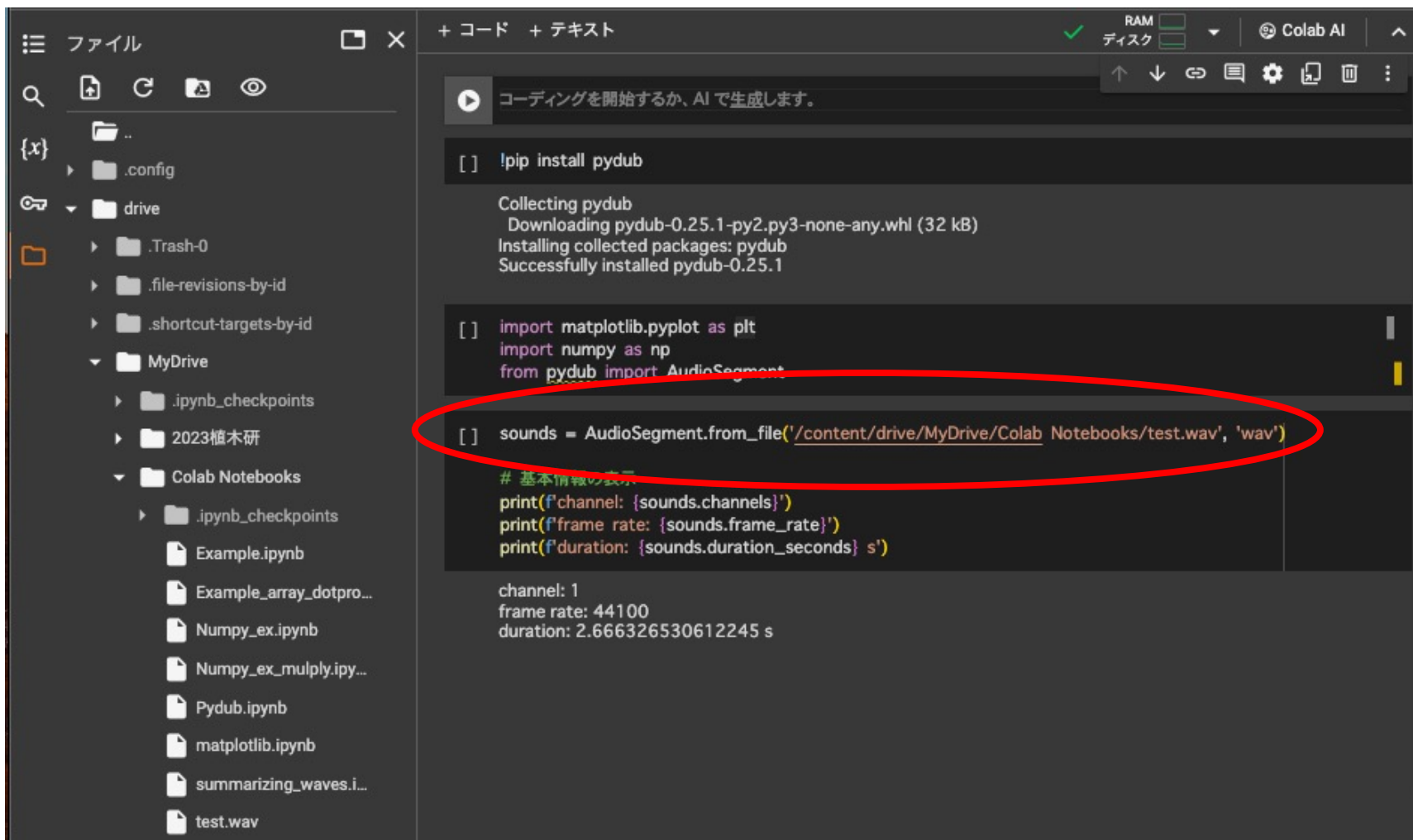
# 基本情報の表示
print(f'channel: {sounds.channels}')
print(f'frame rate: {sounds.frame_rate}')
print(f'duration: {sounds.duration_seconds} s')
```

The output of the second code cell shows the audio file's metadata:

```
channel: 1
frame rate: 44100
duration: 2.666326530612245 s
```

Google Driveに転送したデータをPydubで読み込んでみよう

ファイルの指定にはパスの記載に注意



```

+ コード + テキスト
✓ RAM ディスク Colab AI
↑ ↓ ↺ ⌨ ⚙ 🗑 ⋮

▶ コーディングを開始するか、AI で生成します。

[ ] !pip install pydub

Collecting pydub
  Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Installing collected packages: pydub
Successfully installed pydub-0.25.1

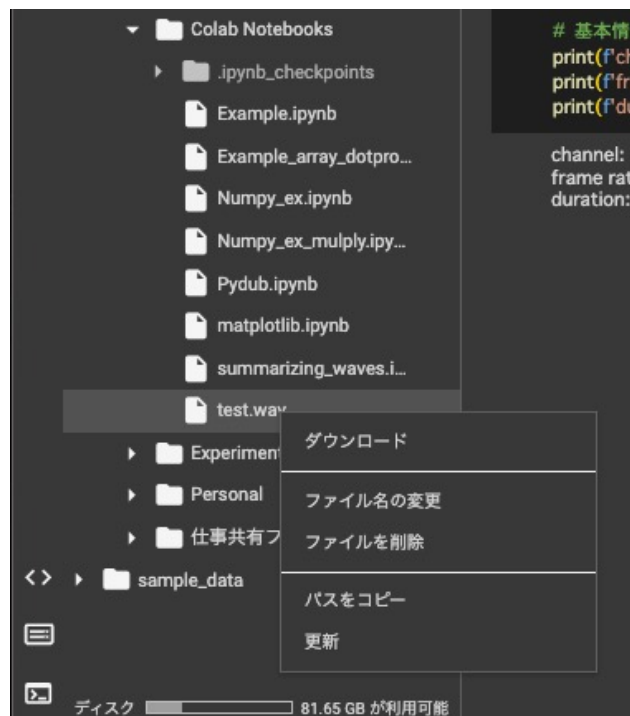
[ ] import matplotlib.pyplot as plt
import numpy as np
from pydub import AudioSegment

[ ] sounds = AudioSegment.from_file('/content/drive/MyDrive/Colab Notebooks/test.wav', 'wav')

# 基本情報の表示
print(f'channel: {sounds.channels}')
print(f'frame rate: {sounds.frame_rate}')
print(f'duration: {sounds.duration_seconds} s')

channel: 1
frame rate: 44100
duration: 2.666326530612245 s
  
```

Google Driveに転送したデータをPydubで読み込んでみよう



- ファイルの指定にはパスの記載に注意
- 読み込みを行いたいパスの取得には「右クリック」→「パスをコピー」
- 取得したパスをペーストして使用するのが簡便

音響データ処理

wavファイルを読み込んで、基本情報を表示させよう

```
[ ] !pip install pydub
```

```
Collecting pydub
  Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Installing collected packages: pydub
Successfully installed pydub-0.25.1
```

```
[ ] import matplotlib.pyplot as plt
import numpy as np
from pydub import AudioSegment
```

```
[ ] sounds = AudioSegment.from_file('/content/drive/MyDrive/Colab Notebooks/test.wav', 'wav')
```

```
# 基本情報の表示
```

```
print(f'channel: {sounds.channels}')
print(f'frame rate: {sounds.frame_rate}')
print(f'duration: {sounds.duration_seconds} s')
```

```
channel: 1
frame rate: 44100
duration: 2.666326530612245 s
```



チャンネル：モノラルなら 1
サンプリング周波数
録音時間

Pythonを使って音響データをフーリエ変換してみよう

```
[6] # ただし,今回の場合はモノラルを想定。つまり,sounds.channels = 1
sig = np.array(sounds.get_array_of_samples())[::sounds.channels]
dt = 1.0/sounds.frame_rate # サンプリング時間

# 時間アレイを作る
tms = 0.0 # サンプル開始時間を0にセット
tme = sounds.duration_seconds # サンプル終了時刻
tm = np.linspace(tms, tme, len(sig), endpoint=False) # 時間ndarrayを作成

# 離散フーリエ変換(DFT)
N = len(sig)
X = np.fft.fft(sig)
f = np.fft.fftfreq(N, dt) # Xのindexに対応する周波数のndarrayを取得

# データをプロット
fig, (ax01, ax02) = plt.subplots(nrows=2, figsize=(6, 8))
plt.subplots_adjust(wspace=0.0, hspace=0.6)

ax01.set_xlim(tms, tme)
ax01.set_xlabel('time (s)')
ax01.set_ylabel('acoustic signal intensity x (-)')
ax01.plot(tm, sig) # 入力信号

ax02.set_xlim(0, 2000)
ax02.set_xlabel('frequency (Hz)')

ax02.set_ylabel('|X|/N (-)')
ax02.plot(f[0:N//2], np.abs(X[0:N//2])/N) # 振幅スペクトル

plt.show()
```

← 離散フーリエ変換
(Numpyを使った実装)

※ Scipy等のライブラリでも同様

Pythonを使って音響データをフーリエ変換してみよう

```
[6] # ただし,今回の場合はモノラルを想定。つまり,sounds.channels = 1
sig = np.array(sounds.get_array_of_samples())[:,sounds.channels]
dt = 1.0/sounds.frame_rate # サンプリング時間

# 時間アレイを作る
tms = 0.0 # サンプル開始時間を0にセット
tme = sounds.duration_seconds # サンプル終了時刻
tm = np.linspace(tms, tme, len(sig), endpoint=False) # 時間ndarrayを作成

# 離散フーリエ変換(DFT)
N = len(sig)
X = np.fft.fft(sig)
f = np.fft.fftfreq(N, dt) # Xのindexに対応する周波数のndarrayを取得

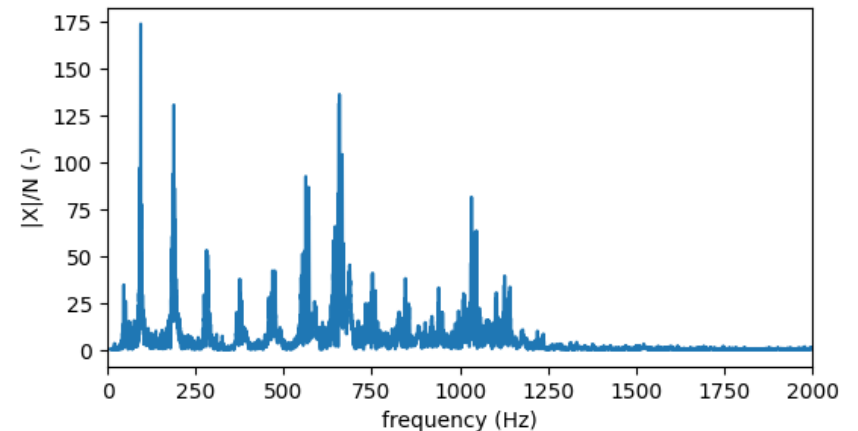
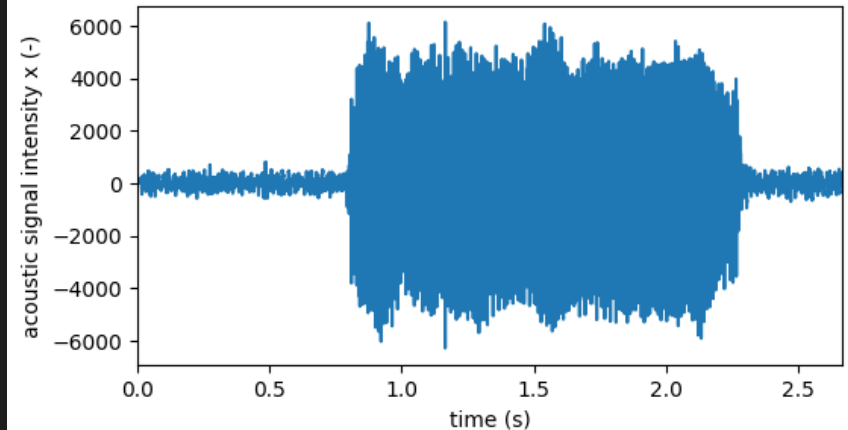
# データをプロット
fig, (ax01, ax02) = plt.subplots(nrows=2, figsize=(6, 8))
plt.subplots_adjust(wspace=0.0, hspace=0.6)

ax01.set_xlim(tms, tme)
ax01.set_xlabel('time (s)')
ax01.set_ylabel('acoustic signal intensity x (-)')
ax01.plot(tm, sig) # 入力信号

ax02.set_xlim(0, 2000)
ax02.set_xlabel('frequency (Hz)')

ax02.set_ylabel('|X|/N (-)')
ax02.plot(f[0:N//2], np.abs(X[0:N//2])/N) # 振幅スペクトル

plt.show()
```



Pythonを使って音響データをフーリエ変換してみよう

```
# ただし,今回の場合はモノラルを想定。つまり,sounds.channels = 1
sig = np.array(sounds.get_array_of_samples())[:sounds.channels]
dt = 1.0/sounds.frame_rate # サンプリング時間

# 時間アレイを作る
tms = 0.0 # サンプル開始時間を0にセット
tme = sounds.duration_seconds # サンプル終了時刻
tm = np.linspace(tms, tme, len(sig), endpoint=False) # 時間ndarrayを作成

# 離散フーリエ変換(DFT)
N = len(sig)
X = np.fft.fft(sig)
f = np.fft.fftfreq(N, dt) # Xのindexに対応する周波数のndarrayを取得

# データをプロット
fig, (ax01, ax02) = plt.subplots(nrows=2, figsize=(6, 8))
plt.subplots_adjust(wspace=0.0, hspace=0.6)

ax01.set_xlim(tms, tme)
ax01.set_xlabel('time (s)')
ax01.set_ylabel('acoustic signal intensity x (-)')
ax01.plot(tm, sig) # 入力信号

ax02.set_xlim(0, 2000)
ax02.set_xlabel('frequency (Hz)')

ax02.set_ylabel('|X|/N (-)')
ax02.plot(f[0:N//2], np.abs(X[0:N//2])/N) # 振幅スペクトル

# Google Colabでグラフをファイルとして保存する
from google.colab import files

plt.savefig("Acoustic_analysis.png")
files.download("Acoustic_analysis.png")
```

Google Colabでプロットした図を保存するには
plt.savefig()だけではできないので注意



ローカルマシンに保存可能

Pythonを使って音響データをフーリエ変換してみよう

練習問題

- 前回Numpyで作成した正弦波の時系列データをフーリエ変換して、正しく処理できているか確認してみよう
- PCM録音で収録した音響データをフーリエ変換してみよう

音響特徴量解析

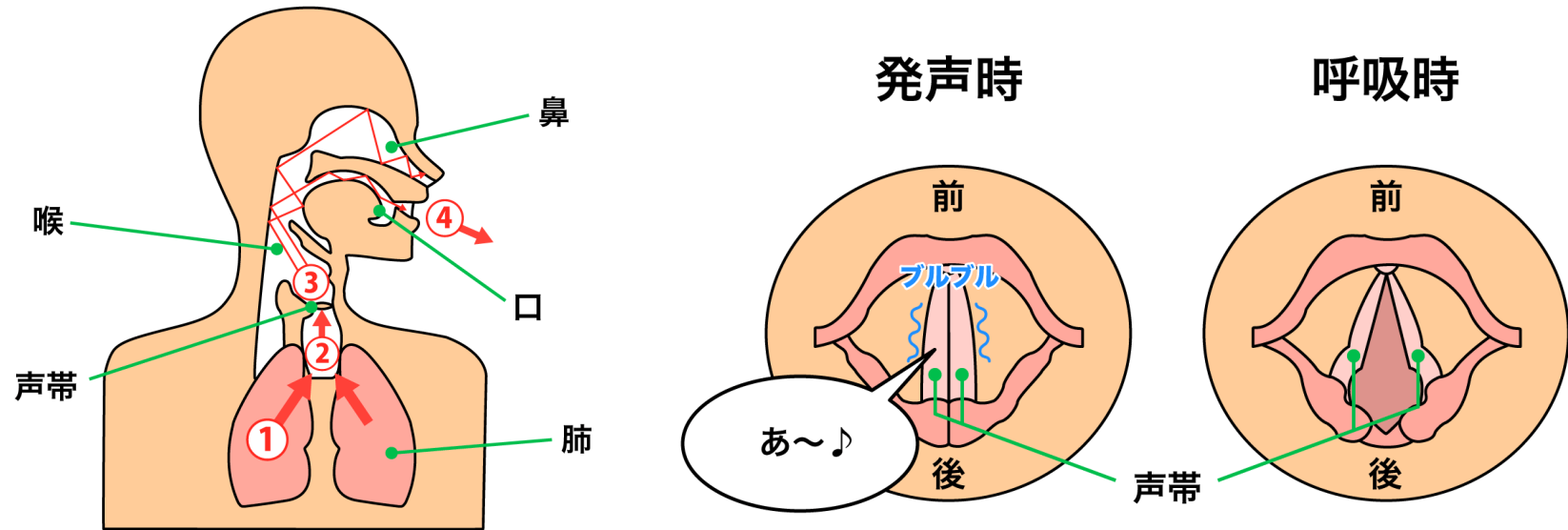
周波数解析手法

- フーリエ変換
(スペクトログラム)
- ウェーブレット変換
(スカログラム)
- ケプストラム分析

ケプストラム分析

- 時系列音圧データ
↓ 離散フーリエ変換
- パワースペクトル
↓ 対数処理 (dB)
- 対数パワースペクトル
↓ 逆フーリエ変換
- ケプストラム
音声信号の畳み込み成分の分離が可能

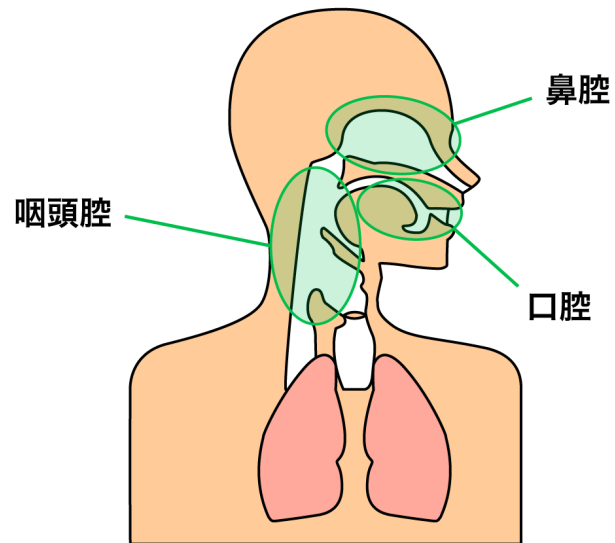
声の発生メカニズム



1. 肺から空気を送る
2. 呼気が声帯を振動させ、音が鳴る（音源）
3. 声帯で発生した音が喉、鼻、口で響く（共鳴腔）
4. 共鳴音を口などの器官で音（母音・子音）を形成

<https://acro-musicschool.com/blog/vocal/vocal001.html>

声の発生メカニズム



共鳴腔

身体の中で声が反響する空間c

母音

口腔内で呼気の流れがあまり妨げられないで発せられる言語音

子音

口腔内で呼気の流れがある程度妨げられ発せられる言語音

<https://acro-musicschool.com/blog/vocal/vocal001.html>

声の発生メカニズム：信号処理の観点から

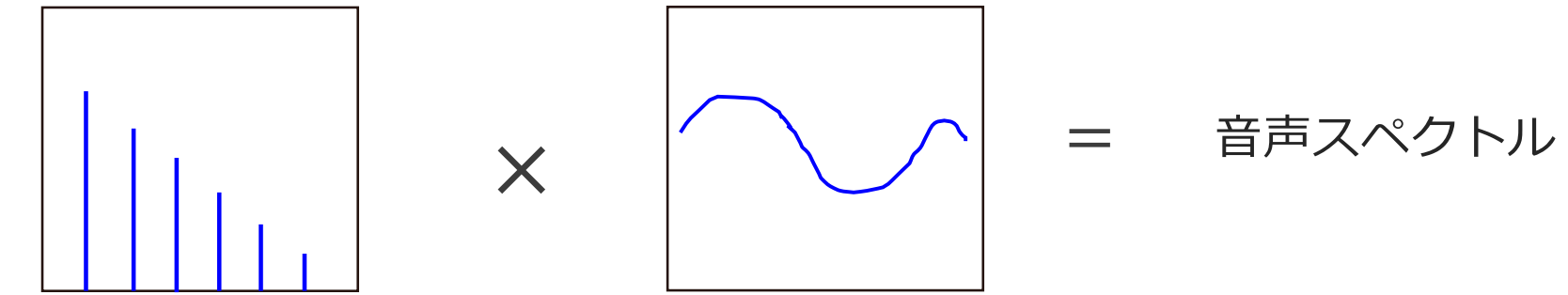
- 時間領域 ※ 音源音圧と声道フィルタの畳み込み

$$a(t) * h(t) = s(t)$$

声帯振動信号 声道フィルタ 音声信号

↓ フーリエ変換

- 周波数領域

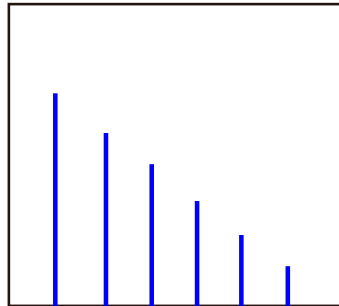


声帯振動スペクトル 声道スペクトル 音声スペクトル

※ 音源スペクトルと声道スペクトルの積

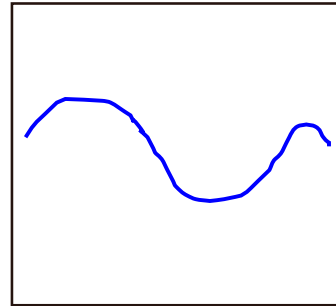
声の発生メカニズム：信号処理の観点から

■ 周波数領域（縦軸真数軸）



声帯振動スペクトル

×



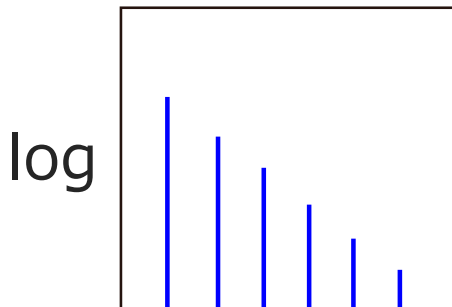
声道スペクトル

=

音声スペクトル

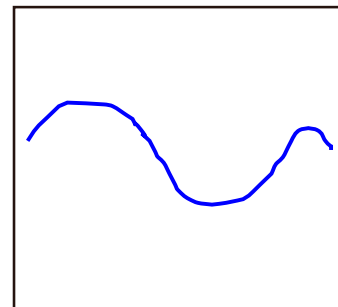
↓ 対数（dB表示）

■ 周波数領域（縦軸対数軸）



声帯振動スペクトル

+



声道スペクトル

=

log 音声スペクトル

ケプストラム分析

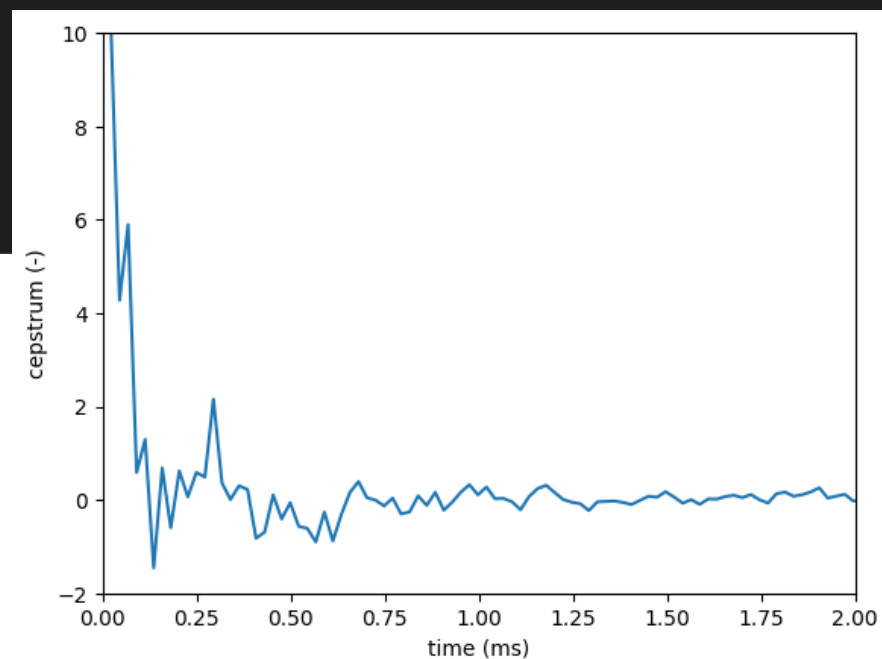
```
X_abs = np.abs(X)
LogAmpX = 10*np.log10(X_abs**2)
```

dB変換

```
cps = np.real(np.fft.ifft(LogAmpX))
```

ケプストラムに変換（逆フーリエ変換）

```
plt.xlim([0, 2])
plt.ylim([-2, 10])
plt.plot(tm*1000, cps)
plt.xlabel("time (ms)")
plt.ylabel("cepstrum (-)")
plt.show()
```



ケフレンシ (quefreny)

ケプストラム分析

```
[52] # ローパスリフタ
# ケプストラムの高次成分を0にして微細構造を除去し、
# 緩やかなスペクトル包絡のみ抽出
cepCoef = 40          # ケプストラム次数
cpsLif = np.array(cps) # arrayをコピー
# 高周波成分を除く(左右対称なので注意)
cpsLif[cepCoef:len(cpsLif) - cepCoef + 1] = 0
```

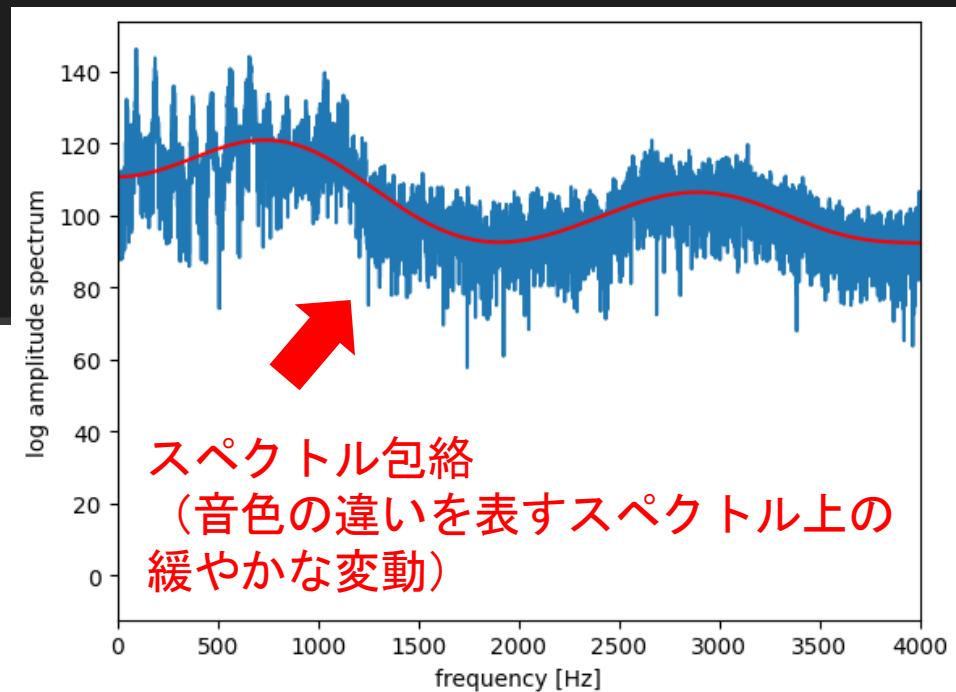
```
# ケプストラム領域をフーリエ変換してスペクトル領域に戻す
# リフタリング後の対数スペクトル
dftSpc = np.fft.fft(cpsLif)

# オリジナルの対数スペクトルを描画
plt.plot(f[0:N//2], LogAmpX[0:N//2])
# 高周波成分を除いた声道特性のスペクトル包絡を重ねて描画
plt.plot(f[0:N//2], dftSpc[0:N//2], color="red")
plt.xlabel("frequency [Hz]")
plt.ylabel("log amplitude spectrum")
plt.xlim(0, 4000)

plt.show()
```

リフタ
※ケプストラム分析におけるフィルタ

※高周波成分を取り除くことが可能



音響解析

練習問題

- PCM録音を使って、自分の声を録音してみよう
- 録音した声をフーリエ変換して周波数スペクトルを解析してみよう
- 録音した声をケプストラム分析してみよう

発展問題

- 録音した音声をスペクトログラム（フーリエ変換の時間一周波数表現）分析してみよう
- ウェーブレット変換してみよう
- スカログラム（ウェーブレット変換の時間一周波数表現）分析をしてみよう

プレゼンテーション課題

**自分自身で音響データを取得、分析、抽出した音響特徴量を用いて
データ分類または予測を行う機械学習モデルを実装し
性能を評価せよ**

※ ただし、機械学習を適用するにあたり狙いを定めて取り組むこと

対象音

- 音声、物理現象にて生じる音など音響データであれば制限はなし
- ただし、自身の研究テーマに直結するデータあるいは現象は扱わない

音響解析手法

- フーリエ変換等の本授業内で取り上げたものに制限しない

機械学習手法

- これから取り上げるニューラルネットワークに限定しない
- 教師あり・なし学習の種別を問わない
- ラベル分類、回帰分析など目的に応じた手法を選択すること



Tokyo University of Science